

Trabajo Práctico 1: Representación de Datos y Aritmética de Computadores

Asignatura: Electrónica Digital (2026)

Carrera: Ingeniería Informática

Institución: Facultad de Ingeniería y Ciencias Agrarias, Universidad Católica Argentina (UCA)

Integrantes: Lucas Di Tomase y Valentín Michellic

Repositorio de Código: <https://github.com/lucasditomase/Trabajo-Practico-1-de-Electronica-Digital>

1. Introducción y Marco Teórico

El presente informe técnico constituye la resolución integral del **Trabajo Práctico 1** de la materia **Electrónica Digital**. El objetivo principal es analizar experimental y analíticamente la forma en que los sistemas digitales y las arquitecturas de procesadores contemporáneas representan e interaccionan con la información a nivel numérico y de memoria real.

1.1 Organización de la Memoria Principal y Endianness

La memoria RAM física se organiza como un arreglo lineal de celdas direccionables, donde la unidad mínima de almacenamiento con dirección propia es el **byte** (8 bits). Cuando un tipo de dato requiere múltiples bytes (por ejemplo, `int` de 32 bits / 4 bytes):

- **Little-Endian:** El byte menos significativo (LSB) se almacena en la dirección de memoria más baja. Es el estándar utilizado por las arquitecturas x86_64 y la mayoría de procesadores ARM.
- **Big-Endian:** El byte más significativo (MSB) se almacena en la dirección de memoria más baja.

1.2 Sistemas de Representación Numérica Entera

1. **Binario Natural (unsigned):** Sistema ponderado en base 2 para enteros no negativos. Para un registro de N bits, el rango abarcado es $[0, 2^N - 1]$.
2. **Complemento a la Base / Complemento a 2 (signed):** Sistema estándar para la representación de enteros con signo. El bit más significativo (MSB) actúa con peso negativo (-2^{N-1}). La gran ventaja del complemento a 2 radica en que permite reutilizar el mismo circuito sumador digital tanto para sumas como para restas sin requerir una Unidad Aritmético-Lógica (ALU) dedicada a la sustracción. El rango abarcado para N bits es $[-2^{N-1}, 2^{N-1} - 1]$.

1.3 Estándar IEEE 754 de Punto Flotante (32 bits)

Para la representación de números reales de precisión simple, la norma IEEE 754 define un patrón binario de 32 bits divididos en tres campos:

$$V = (-1)^s \times (1 + m) \times 2^{e-127}$$

- **Bit de Signo (s, bit 31):** 0 representa valores positivos; 1 representa valores negativos.

- **Exponente Polarizado (e, bits 30-23):** Campo de 8 bits con un sesgo (*bias*) de 127. El exponente real es $E = e - 127$.
- **Mantisa / Fracción (m, bits 22-0):** Campo de 23 bits fraccionarios. En números normalizados, el bit entero 1. es implícito (*hidden bit*), lo que otorga 24 bits efectivos de precisión.

2. Desarrollo de Puntos Obligatorios (Puntos 1 al 5)

Punto 1: Representación Interna, Rangos y Memoria Real (char, int, unsigned char, unsigned int)

Fundamento Teórico y Análisis

El tipo de dato determina la cantidad de bytes asignados y la interpretación del patrón de bits. En una arquitectura de 32/64 bits Little-Endian:

- `char` (1 byte, con signo): Rango $[-128, 127]$.
- `unsigned char` (1 byte, sin signo): Rango $[0, 255]$.
- `int` (4 bytes, con signo): Rango $[-2.147.483.648, 2.147.483.647]$.
- `unsigned int` (4 bytes, sin signo): Rango $[0, 4.294.967.295]$.

Código Fuente en C (ISO C99)

```
static void inspeccionar_memoria(const char *etiqueta, const void *ptr, size_t tamano)
{
    const uint8_t *bytes = (const uint8_t *)ptr;
    printf("Variable '%s' (%zu byte/s):\n", etiqueta, tamano);

    printf(" Hexadecimal en memoria (LSB -> MSB): ");
    for (size_t i = 0; i < tamano; i++) {
        printf("0x%02X ", bytes[i]);
    }
    printf("\n Representación binaria (MSB -> LSB): ");
    for (int i = (int)tamano - 1; i >= 0; i--) {
        for (int b = 7; b >= 0; b--) {
            printf("%d", (bytes[i] >> b) & 1);
        }
        printf(" ");
    }
    printf("\n\n");
}
```

Salida por Consola (stdout)

```
char:          1 byte(s) | Rango: [-128, 127]
unsigned char: 1 byte(s) | Rango: [0, 255]
int:           4 byte(s) | Rango: [-2147483648, 2147483647]
unsigned int:   4 byte(s) | Rango: [0, 4294967295]

Variable 'char c = -42' (1 byte/s):
  Hexadecimal en memoria (LSB -> MSB): 0xD6
  Representación binaria (MSB -> LSB): 11010110

Variable 'unsigned char uc = 200' (1 byte/s):
  Hexadecimal en memoria (LSB -> MSB): 0xC8
  Representación binaria (MSB -> LSB): 11001000

Variable 'int i = -12345678' (4 byte/s):
  Hexadecimal en memoria (LSB -> MSB): 0xB2 0x9E 0x43 0xFF
  Representación binaria (MSB -> LSB): 11111111 01000011 10011110 10110010

Variable 'unsigned int ui = 3000000000U' (4 byte/s):
  Hexadecimal en memoria (LSB -> MSB): 0x00 0x5E 0xD0 0xB2
  Representación binaria (MSB -> LSB): 10110010 11010000 01011110 00000000
```

Análisis de Bits

1. Para `char c = -42` : En 8 bits complemento a 2, $+42_{10} = 00101010_2$. Su complemento a 1 es 11010101_2 , y sumando 1 obtenemos $11010110_2 = 0xD6$.
2. Para `int i = -12345678` : En hexadecimal de 32 bits es `0xFF439EB2`. Debido al orden Little-Endian, la memoria almacena los bytes en la dirección ascendente como `0xB2 0x9E 0x43 0xFF`.

Punto 2: Representación Interna de Flotantes (Estándar IEEE 754)

Fundamento Teórico

Tomemos el valor $f = -12.375_{10}$:

1. Conversión binaria: $12_{10} = 1100_2$, $0.375_{10} = 0.011_2 \Rightarrow 1100.011_2$.
2. Normalización binaria: $1.100011_2 \times 2^3$.
3. Signo: Negativo $\Rightarrow s = 1$.
4. Exponente: $E = 3 \Rightarrow e = 3 + 127 = 130_{10} = 10000010_2$.
5. Mantisa: Parte fraccionaria $.100011_2$, rellena con ceros a la derecha hasta completar 23 bits: 10001100000000000000000 .
6. Palabra de 32 bits: $1\ 10000010\ 10001100000000000000000 \Rightarrow 0xC1460000$.

Salida por Consola (stdout)

```
Tamaño del float: 4 bytes (32 bits)
Mínimo positivo normalizado (FLT_MIN): 1.175494E-38
Máximo representable (FLT_MAX): 3.402823E+38
Dígitos de precisión decimal (FLT_DIG): 6

Analizando el valor float f = -12.375000:
Patrón de bits completo (Hexadecimal): 0xC1460000
1. Bit de Signo (Bit 31): 1 (1 -> Negativo)
2. Exponente Polarizado (Bits 30-23): 130 (0x82) -> Exponente Real E = 130 - 127 = 3
3. Mantisa/Fracción (Bits 22-0): 0x460000
   Bits de mantisa en binario: 100011000000000000000000
4. Reconstrucción teórica:  $(-1)^1 * (1 + 0.5 + 0.03125) * 2^3 = -12.3750$ 
```

Punto 3: Mecanismo de Extensión de Signo (char a int)

Fundamento Teórico y Análisis

Al asignar una variable de menor ancho con signo a una palabra más ancha, la CPU replica el bit MSB (bit de signo) a lo largo de los bits de mayor peso. Esto preserva la equivalencia de valor en la aritmética de complemento a 2.

- Para $c = -5$ (0xFB / 11111011), la extensión a 32 bits rellena los 24 bits superiores con 1 $\Rightarrow$ 0xFFFFFFF8 = -5_{10} .
- Para $c = +5$ (0x05 / 00000101), se rellena con 0 $\Rightarrow$ 0x00000005 = $+5_{10}$.

Salida por Consola (stdout)

```
Caso Negativo (c_neg = -5):
char (8 bits): Hex 0xFB | Binario: 11111011
int (32 bits): Hex 0xFFFFFFF8 | Decimal: -5 | MSB propagado a 1

Caso Positivo (c_pos = 5):
char (8 bits): Hex 0x05 | Binario: 00000101
int (32 bits): Hex 0x00000005 | Decimal: 5 | MSB propagado a 0
```

Punto 4: Comportamiento ante Desbordamiento (Overflow / Wrap-around)

Fundamento Teórico

En el lenguaje C no existen excepciones en tiempo de ejecución ante un desbordamiento entero.

- **unsigned int**: Rige la aritmética modular estricta módulo 2^{32} . Sumar 1 a 0xFFFFFFFF produce 0x00000000 (0).
- **int con signo**: Al sumar 1 a INT_MAX (0x7FFFFFFF), el bit MSB cambia a 1 (0x80000000), el cual corresponde a INT_MIN (-2.147.483.648).

- **Detección en Hardware:** La ALU detecta la condición de desbordamiento en enteros con signo comparando el acarreo entrante (C_{in}) y el acarreo saliente (C_{out}) del bit MSB:

$$\text{Overflow} = C_{in} \oplus C_{out}$$

Salida por Consola (stdout)

```
Entero Sin Signo (unsigned int):
  Valor Máximo (UINT_MAX): 4294967295 (Hex: 0xFFFFFFFF)
  UINT_MAX + 1:           0 (Hex: 0x00000000) -> Wrap-around a 0

Entero Con Signo (int en Complemento a 2):
  Valor Máximo (INT_MAX): 2147483647 (Hex: 0x7FFFFFFF)
  INT_MAX + 1:           -2147483648 (Hex: 0x80000000) -> Wrap-around a INT_MIN
```

Punto 5: Pérdida de Precisión en float (Cancelación y Alineación)

Fundamento Teórico y Análisis

Para realizar la adición de dos flotantes en IEEE 754, la FPU debe alinear las mantisas desplazando la mantisa del número con menor exponente hacia la derecha. En la operación $10^7 + 0.1$:

- $10^7 \approx 2^{23.25}$, por lo que su exponente es $E = 23$.
- $0.1 \approx 2^{-3.32}$, por lo que su exponente es $E = -4$. La diferencia entre exponentes es $23 - (-4) = 27$ posiciones. Debido a que la mantisa de `float` tiene solo 23 bits explícitos (24 efectivos), desplazar la mantisa de 0.1 hacia la derecha 27 lugares provoca que todos sus bits significativos caigan fuera de la ventana de precisión y sean totalmente descartados. Por lo tanto, $10.000.000,0 + 0,1 = 10.000.000,0$.

Salida por Consola (stdout)

```
Operando Grande: 10000000.00
Operando Pequeño: 0.1000
Resultado Suma: 10000000.00

¡Demostrado! (grande + pequeño == grande).
Explicación: Para realizar la suma, la FPU debe alinear los exponentes.
Debido a que la diferencia de exponentes supera los 24 bits de mantisa,
los bits del operando menor son desplazados completamente fuera de la ventana de preci:
```

3. Desarrollo de Experimentos de IA (IA1 al IA6)

IA1: Inexactitud en Punto Flotante y Comparaciones

- **Comportamiento:** La representación de 0.1_{10} en binario es una fracción periódica pura ($0.0001100110011\dots_2$). Al sumarlo 10 veces, se acumula un error de redondeo que resulta en 1.0000001192 ,

haciendo que `suma == 1.0f` sea **FALSO**.

- **Solución:** Utilizar una diferencia absoluta contra una tolerancia permisible ($\epsilon = 10^{-6}$).

```
Suma acumulada de 0.1f diez veces: 1.0000001192
Comparación directa (suma == 1.0f): FALSO
Comparación con EPSILON (1e-6f): VERDADERO (Correcto en ingeniería)
```

IA2: Comparaciones Mixtas (signed VS unsigned)

- **Comportamiento:** Por las reglas de conversión implícita de C (*Usual Arithmetic Conversions*), al comparar `int` con `unsigned int`, el `int` es promovido a `unsigned int`. El valor `-1` (`0xFFFFFFFF`) pasa a ser `4294.967.295`, de modo que `-1 < 1U` evalúa como **FALSO**.
- **Solución:** Aplicar un casting explícito `negativo < (int)uno`.

```
Evaluando expresión en C: -1 < 1U
Resultado C: -1 < 1U es FALSO
Explicación: Por las reglas 'Usual Arithmetic Conversions', -1 (0xFFFFFFFF)
es promovido a unsigned int, reorientando su valor a 4294967295.
Solución con casting (-1 < (int)1U): VERDADERO (Comparación correcta en con signo)
```

IA3: División Entera y Truncamiento (ISO C99)

- **Comportamiento:** La norma C99 especifica que la división entera trunca **hacia cero**.
- **Identidad:** $(a / b) * b + (a \% b) = a$. Esta ecuación impone que el signo del resto $(a \% b)$ coincida obligatoriamente con el signo del dividendo a .

```
Resultados observados:
 5 / 2 = 2 | 5 % 2 = 1
(-5) / 2 = -2 | (-5) % 2 = -1
 5 / (-2) = -2 | 5 % (-2) = 1
(-5) / (-2) = 2 | (-5) % (-2) = -1

Comprobación de Identidad Teórica: (a / b) * b + (a % b) == a
Para a = -5, b = 2 -> (-2 * 2) + (-1) = -5 (Cumple: SI)
```

IA4: Multiplicación y Promoción de Tipos

- **Comportamiento:** En la expresión `long long res = a * b`, la multiplicación entre dos `int` de 32 bits se calcula dentro del registro intermedio de la ALU de 32 bits *antes* de realizar la asignación. Al superar $2^{31} - 1$, se produce desbordamiento.
- **Solución:** Castear al menos un operando previo a la operación: `(long long)a * b`.

```
Variables: a = 100000, b = 100000 (Producto real: 10,000,000,000)
Sin casting previa: (long long)(a * b) = 1410065408 (ERROR por overflow 32 bits)
Con casting previa: (long long)a * b = 10000000000 (CORRECTO)
```

IA5: Asimetría del Complemento a la Base e `INT_MIN`

- **Comportamiento:** En complemento a 2 con 32 bits, el rango es $[-2.147.483.648, 2.147.483.647]$. El valor positivo $+2^{31}$ no puede ser representado.
- **Efecto de Negación:** `INT_MIN` tiene el patrón `0x80000000`. Su complemento a 1 es `0x7FFFFFFF`, y al sumar 1 regresa exactamente al mismo patrón `0x80000000`. Por ende, `-INT_MIN` y `abs(INT_MIN)` continúan dando `INT_MIN`.

```
Rango de int (32 bits): [-2147483648, 2147483647]
Valor INT_MIN:         -2147483648 (Hex: 0x80000000)
abs(INT_MIN):          -2147483648 (Hex: 0x80000000) -> Asimetría de Rango
-INT_MIN:              -2147483648 (Hex: 0x80000000)
Explicación: En 32 bits complemento a 2, no existe la representación de  $+2^{31}$  (2147483648). El complemento a 1 de 0x80000000 es 0x7FFFFFFF, que al sumar 1 vuelve a ser 0x80000000.
```

IA6: Detección de Acarreo (`carry`) en Software

- **Comportamiento:** En la adición de dos enteros sin signo `a + b`, si ocurre un desbordamiento por acarreo (`carry`), la suma resultante es strictly menor que cualquiera de los dos operandos individuales.
- **Condición:** `(a + b) < a` permite detectar con exactitud matemática la presencia de acarreo en software sin necesidad de acceder a los registros de flags del procesador.

```
Operando a: 4294967290 (Hex: 0xFFFFF8FA)
Operando b: 10 (Hex: 0x0000000A)
Suma (a + b): 4 (Hex: 0x00000004)
Condición de Acarreo ((a + b) < a): VERDADERO -> OCURRIÓ CARRY
¡Demostrado! Se detectó el acarreo en software sin consultar registros de banderas de
```

4. Conclusiones Técnicas

1. **Comprensión del Hardware y Memoria:** Los tipos de datos en C no son meras abstracciones matemáticas, sino representaciones físicas ligadas al ancho de palabra, registros de la ALU y esquemas de direccionamiento (Little-Endian) del hardware subyacente.
2. **Seguridad y Desarrollo de Firmware:** Comprender fenómenos como la asimetría de `INT_MIN`, la extensión de signo y las promociones implícitas de tipos es fundamental para prevenir vulnerabilidades de desbordamiento (*integer overflow*) y comportamiento indefinido (*Undefined Behavior*) en código embebido crítico y controladores de dispositivos.

3. **Punto Flotante vs. Precisión Fija:** Las limitaciones del estándar IEEE 754 (errores de redondeo acumulativos y pérdida de precisión por alineación de mantisas) imponen que en aplicaciones donde la precisión absoluta sea crítica (por ejemplo, contabilidad financiera o control numérico de alta exactitud) se prefiera la aritmética de punto fijo o librerías de precisión arbitraria en lugar de flotantes nativos.